

LAB_1 - Revision of Algorithms' Fundamentals

A2 - S4 - 2025/2026 - II.2415

Advanced Algorithms and Programming

Team 04

Damiano Spadaccini - Nicolò Romanelli

GitHub repository: <https://github.com/damspada/AAP-T04-Semester-Project> (branch LAB_01_Revision, folder LAB_01)

This is the final version of our LAB_1 report. We kept the pseudo-code of the report written during the session (T04_LAB1_AES) and for each exercise we added more complexity analysis, a short reflection and the tests we did on our Python code. Where the first version had a mistake, we wrote a short note.

The code is on GitHub (link above). Each file can be run alone and prints the result of every test.

EXERCISE 1 - INTEGER MIRROR

Function ReverseDigits(n: Integer) -> Integer

```
mirror <- 0
While n > 0 Do
    // take the last digit
    digit <- n mod 10
    // add it to the end of the result
    mirror <- mirror * 10 + digit
    // remove it from n
    n <- n div 10
End While
Return mirror
```

End

1. How many arithmetic operations are performed for a d-digit number?

Each turn of the loop does 4 arithmetic operations: one mod, one multiplication, one addition and one div. The loop runs once for each digit, so we have $4d$ arithmetic operations. If we also count the test $n > 0$, which is done $d + 1$ times (the last time it is false), the total is $5d + 1$. For example 315 has $d = 3$, so 12 arithmetic operations and 16 in total.

2. What is the time complexity (number of expensive operations) in terms of the input value n?

The expensive operations are div and mod, and we do one of each for every digit. A number n has $\text{floor}(\log_{10}(n)) + 1$ digits, so the complexity is $\Theta(\log n)$: every turn divides n by 10.

More complexity analysis and reflection

The loop always runs once for each digit, so the best and the worst case are the same. For $n = 0$ it does not run at all and the result is 0, which is correct, and in general the loop always stops, because $n \text{ div } 10$ is smaller than n when $n > 0$. The memory is $O(1)$, since we only use mirror, digit and n .

In Python integers have no limit, but in C or Java with a 32-bit int the result could overflow: 1 000 000 009 fits, but its mirror 9 000 000 001 does not. Negative numbers are not allowed, so in our code we print an error and return -1, which can never be a real result because a reversed number is never negative.

Test set

Case	n	Expected	Got
example 1	315	513	513
trailing zeros	400	4	4
single digit	7	7	7
zero	0	0	0
zeros in the middle	10203	30201	30201
negative number	-12	-1	-1

EXERCISE 2 - BALANCED SYMBOL CHECKER

Function IsBalanced(s: String) -> Boolean

stack <- empty stack

For each char in s Do

// the three opening symbols are pushed and wait for their closer

If char = "(" Or char = "[" Or char = "{" Then

Push(stack, char)

// a closer needs its own opener on top of the stack

Else If char = ")" Then

If IsEmpty(stack) Then

Return False

End If

top <- Pop(stack)

If top != "(" Then

Return False

End If

Else If char = "]" Then

If IsEmpty(stack) Then

Return False

End If

top <- Pop(stack)

If top != "[" Then

Return False

End If

Else If char = "}" Then

If IsEmpty(stack) Then

Return False

End If

top <- Pop(stack)

If top != "{" Then

Return False

End If

End If

End For

// a leftover opener means the string is unclosed

Return IsEmpty(stack)

End

1. How many comparisons for an n-character string?

At most 8 per character, plus 1 for the final IsEmpty, so at most $8n + 1$. A "(" costs 1 comparison, "[" 2 and "{" 3. A ")" costs 6 (the 3 opening tests, 1 to find its branch, IsEmpty and the match), "]" costs 7 and

"}" 8. Any other character costs 6 and is ignored.

In our code every comparison goes through a small function `compare()` that adds 1 to a counter, and the numbers are the same (last column of the test table). A balanced string can not reach $8n$, because every closer needs an opener before it. The worst one is "{}{}{}...": $3 + 8 = 11$ comparisons every 2 characters, so $5.5n + 1$. For example with $n = 10000$ our code counts 55001.

2. What if we extend to more symbol pairs or other characters?

The complexity stays $\Theta(n)$, only the constant grows: with p pairs a character costs up to $2p + 2$ comparisons. Other characters are ignored, so the checker works also on a line of code (last test). With many pairs it would be better to use a dictionary closer $\rightarrow$ opener, so every character costs the same.

More complexity analysis and reflection

In the worst case the time is $\Theta(n)$, because every character is read once and push and pop are $O(1)$. In the best case the string starts with a closer and we stop immediately. The memory is $O(n)$, because the stack can hold all the characters, for example with "((((". With only one kind of bracket a simple counter would be enough, but with three kinds we have to remember the order of the openers, and this is why we need the stack.

Test set

Case	s	Expected	Got	Comparisons
example 1	"({[]})"	True	True	28
wrong nesting	"([I])"	False	False	9
empty string	""	True	True	1
missing closer	"(((("	False	False	16
closer before opener	")("	False	False	5
line of code	"if (a[i] > 0) { x = f(y); }"	True	True	149

EXERCISE 3 - MERGE OVERLAPPING INTERVALS

Function MergeOverlappingIntervals(L : list of intervals) -> list of intervals

```
n <- length(L)
// empty case
If n = 0 Then
    Return L
End If
// sort for an easy manage
Sort L by start value
// position of the current merged block
last <- 0
For i <- 1 To n-1 Do
    // ask if the intervals overlap
    If L[i].start <= L[last].end Then
        // stretch the block
        L[last].end <- max(L[last].end, L[i].end)
    Else
        last <- last + 1
        L[last] <- L[i]
    End If
End For
// the merged intervals are the first last + 1 cells
Return L[0 .. last]
End
```

Correction from the session report: we wrote "Return L", but after the loop the cells after "last" still contain old intervals. For example 1 the list would be $[[1,6], [8,10], [15,18], [15,18]]$. Only $L[0 .. last]$ must be returned.

1. How many comparison operations are needed for n intervals?

Sorting: about $n \log_2(n)$ comparisons, like MergeSort. Merging: one test for every interval after the first one ($n - 1$), plus one inside max when they overlap, so at most $2(n - 1)$. In total at most $n \log_2(n) + 2(n - 1)$. With $n = 1000$ this is about $10\,000 + 2\,000$, so most of the work is the sort.

2. What is the time complexity of your solution?

$O(n \log n)$ for the sort plus $O(n)$ for the merge, so $O(n \log n)$.

More complexity analysis and reflection

One pass is enough because, after sorting, if an interval starts after the end of the current block, all the next ones start even later, so that block is finished. When the list is already sorted, Python's sort (Timsort) is $O(n)$, so in this best case the whole function is $O(n)$.

The merge works directly inside L with only two indexes, so it uses $O(1)$ memory, and only the sort needs $O(n)$ extra memory. Since L is modified, if the original list is still needed it has to be copied before calling the function.

Test set

Case	L	Expected	Got
example 1	[[1, 3], [2, 6], [15, 18], [8, 10]]	[[1, 6], [8, 10], [15, 18]]	[[1, 6], [8, 10], [15, 18]]
touching	[[1, 4], [4, 5]]	[[1, 5]]	[[1, 5]]
not sorted	[[1, 4], [0, 4]]	[[0, 4]]	[[0, 4]]
empty list	[]	[]	[]
one interval	[[3, 7]]	[[3, 7]]	[[3, 7]]
no overlaps	[[1, 2], [4, 5], [7, 8]]	[[1, 2], [4, 5], [7, 8]]	[[1, 2], [4, 5], [7, 8]]
inside another	[[1, 10], [2, 3], [4, 5]]	[[1, 10]]	[[1, 10]]

EXERCISE 4 - POLYNOMIAL EVALUATION

```
Function HornerEval(c: Array of Real, n: Integer, x: Real) -> Real
    result <- 0
    // walk from the highest-degree coefficient down to the constant term
    For i <- n DownTo 0 Do
        result <- result * x + c[i]
    End For
    Return result
End
```

1. How many multiplications for a degree-n polynomial?

With our pseudo-code the loop runs $n + 1$ times, so $n + 1$ multiplications and $n + 1$ additions. The first multiplication is useless, because it computes $0 * x$. If we start with `result <- c[n]` and loop from $n - 1$ down to 0, we get exactly n multiplications and n additions, which is the minimum for a general polynomial of degree n .

Correction from the session report: we wrote "exactly n ", but this is true only with the small change described above.

2. Comparison with the naive approach: what is the improvement?

The naive way computes every power x^i from zero: $0 + 1 + \dots + n = n(n+1)/2$ multiplications, plus $n + 1$ to multiply by the coefficients, so $(n+1)(n+2)/2$, which is quadratic. Keeping the last power and multiplying it by x each time gives about $2n$, still more than Horner.

Degree 19: Horner needs 19 multiplications (20 with our pseudo-code), the naive way $190 + 20 = 210$.

Degree 200: Horner needs 200 multiplications (201 with our pseudo-code), the naive way $20\,100 + 201 = 20\,301$, about 100 times more.

More complexity analysis and reflection

Horner is $\Theta(n)$ in every case, because the loop always does $n + 1$ steps and there is no way to stop earlier, and it uses $O(1)$ memory, only the variable `result`. The gain against the naive method grows with the degree: the ratio between the two counts is $(n + 2) / 2$, so it grows linearly with n .

About numerical precision, floats in Python have only about 16 significant digits, so the result is not always exact. For example with `c = [0.1, 0.2]` and `x = 3` we get 0.7000000000000001 instead of 0.7, and for this reason our tests do not compare floats with `=`, but accept a difference smaller than 10^{-9} . Another case is $P(x) = 1 + x^2$ at $x = 10^8$: with integers Python gives exactly 10000000000000001, while with floats we get $1e+16$, because the $+ 1$ is beyond the 16 digits. Horner helps a bit, because it does fewer operations than the naive method (so fewer rounding errors) and it never computes x^n alone, but it can not remove the problem.

Test set

Case	Coefficients	x	Expected	Got
example	[3, -2, 0, 5]	2.0	39.0	39.0
constant	[7]	100.0	7.0	7.0
empty list	[]	5.0	0	0
x = 0	[4, 9, 9, 9]	0.0	4.0	4.0
decimal coefficients	[0.1, 0.2]	3.0	0.7	0.7000000000000001
big integer x	[1, 0, 1]	100000000	100000000000000001	100000000000000001

EXERCISE 5 - ARRAY ROTATION OPTIMIZATION

1 - TEMPORARY ARRAY

Function RotateTempArray(V: Array, n: Integer, k: Integer)

 If n = 0 Then

 Return

 End If

 k <- k mod n

 temp <- new array of size n

 // the element at index i belongs at index (i + k) mod n, the mod bringing back to the front whatever passes the end

 For i <- 0 To n - 1 Do

 temp[(i + k) mod n] <- V[i]

 End For

 // copy the result back into V

 For i <- 0 To n - 1 Do

 V[i] <- temp[i]

 End For

End

2 - ONE BY ONE

Function ShiftRightByOne(V: Array, n: Integer)

 // the last cell is about to be overwritten, so save it first

 last <- V[n - 1]

 // copy each cell from its left neighbour, walking from the end towards the front: going the other way would overwrite a cell before its old value has been copied further right

 i <- n - 1

 While i >= 1 Do

 V[i] <- V[i - 1]

 i <- i - 1

 End While

 // the saved cell wraps around to the front

 V[0] <- last

End

Function RotateOneByOne(V: Array, n: Integer, k: Integer)

 If n = 0 Then

 Return

 End If

 k <- k mod n

 For j <- 1 To k Do

 ShiftRightByOne(V, n)

```
End For
End
```

3 - REVERSE SEGMENTS

```
Function Reverse(V: Array, left: Integer, right: Integer)
    // walk the two ends towards each other, swapping as they go
    While left < right Do
        temp <- V[left]
        V[left] <- V[right]
        V[right] <- temp
        left <- left + 1
        right <- right - 1
    End While
End
```

```
Function RotateReversal(V: Array, n: Integer, k: Integer)
    If n = 0 Then
        Return
    End If
    k <- k mod n
    If k = 0 Then
        Return
    End If
    Reverse(V, 0, n - 1)
    Reverse(V, 0, k - 1)
    Reverse(V, k, n - 1)
End
```

Example of (3) with $V = [1, 2, 3, 4, 5, 6, 7]$ and $k = 10 \bmod 7 = 3$: reverse all gives $[7, 6, 5, 4, 3, 2, 1]$, reverse the first 3 gives $[5, 6, 7, 4, 3, 2, 1]$, reverse the last 4 gives $[5, 6, 7, 1, 2, 3, 4]$.

1. What is the optimal time complexity achievable?

Theta(n): in general every element has to move, so we can not do better than n operations. Approach (3) reaches Theta(n) with only Theta(1) extra space.

2. How does k affect the performance?

Not at all for (1) and (3), which are Theta(n) for every k once k is reduced mod n. Only (2) depends on k: Theta(n * k), so Theta(n²) when k is around n/2 or bigger (with $k < n$ after the mod).

3. When would each approach be preferable?

(3) almost always: linear time and constant memory. (1) when memory is not a problem and we want simple code, or when V must not change (then we just return temp and skip the copy back). (2) only for $k = 1$ or very small k, where it is just one shift.

More complexity analysis and reflection

If we count also the assignments to temp and last, (1) does $2n$ assignments, (2) $k(n + 1)$ and (3) about $3n$. So (1) does a bit less work than (3), but it needs n extra cells, while (2) quickly becomes very slow: with $n = 2000$ and $k = 1999$ it needs about 4 000 000 assignments, against a few thousand for the other two.

The $k \bmod n$ at the beginning is important. With $k = 4001$ and $n = 2000$ it becomes 1, so (2) does one shift instead of 4001, and without it (3) would go outside the array in `Reverse(V, 0, k - 1)`. In Python $k \bmod n$ is between 0 and $n - 1$ also when k is negative, so $k = -1$ gives a rotation to the left, as in the last test.

Test set

Every test was run on the three versions and all three gave the same result.

Case	V	k	Expected	Got
example 1 ($k > n$)	[1, 2, 3, 4, 5, 6, 7]	10	[5, 6, 7, 1, 2, 3, 4]	[5, 6, 7, 1, 2, 3, 4]
one element	[1]	5	[1]	[1]
empty array	[]	3	[]	[]
$k = 0$	[1, 2, 3]	0	[1, 2, 3]	[1, 2, 3]
$k = n$	[1, 2, 3, 4]	4	[1, 2, 3, 4]	[1, 2, 3, 4]
negative k	[1, 2, 3, 4, 5]	-1	[2, 3, 4, 5, 1]	[2, 3, 4, 5, 1]

EXERCISE 6 - FIRST UNIQUE CHARACTER FINDER

1 - TWO PASS

```
Function FirstUniqueTwoPass(s: String, n: Integer) -> Integer
    frequency <- empty dictionary
    // pass 1: count the occurrences of every character
    For i <- 0 To n - 1 Do
        If ContainsKey(frequency, s[i]) Then
            frequency[s[i]] <- frequency[s[i]] + 1
        Else
            frequency[s[i]] <- 1
        End If
    End For

    // pass 2: return the first character whose count is exactly 1
    For i <- 0 To n - 1 Do
        If frequency[s[i]] = 1 Then
            Return i
        End If
    End For
    Return -1
End
```

2 - SINGLE PASS

```
Function FirstUniqueOrdered(s: String, n: Integer) -> Integer
    firstIndex <- empty ordered dictionary

    // one pass: record the first index, mark a repeat with -1
    For i <- 0 To n - 1 Do
        If ContainsKey(firstIndex, s[i]) Then
            firstIndex[s[i]] <- -1
        Else
            firstIndex[s[i]] <- i
        End If
    End For

    // the keys come back in insertion order, so the first surviving
    // index is already the leftmost unique character
    m <- KeyCount(firstIndex)
    For j <- 0 To m - 1 Do
        key <- KeyAt(firstIndex, j)
        If firstIndex[key] != -1 Then
            Return firstIndex[key]
        End If
    End For
    Return -1
End
```

End

1. What is the space complexity of the solution?

$O(\min(n, \text{Sigma}))$, where Sigma is the number of possible characters: the dictionary has at most one key for each different character. With only the 26 lowercase letters it is $O(1)$; with any Unicode character it can become $O(n)$.

2. How many passes through the string are needed?

Approach (1) makes two passes over the string, so at most $2n$ lookups (pass 2 can stop early). Approach (2) makes one pass over the string and then a loop on the keys of the dictionary. The keys are at most $\min(n, \text{Sigma})$, as in question 1, so with a small alphabet this second loop does not depend on n , and in any case it is never longer than the string. (2) is better when the string can be read only once, for example a stream.

3. What data structure is most efficient for this problem?

In general a dictionary (hash table), because lookup and update are $O(1)$ on average, for any character. If the alphabet is small and known, a fixed-size array indexed by the character code is even better: no hashing and always $O(1)$ space.

Correction from the session report: in the first version the last sentence was written two times.

More complexity analysis and reflection

Both approaches are $\Theta(n)$ also in the best case, because we can say that a character is unique only after reading the whole string. Comparing every character with all the others would instead be $\Theta(n^2)$. Approach (2) is correct because when a character comes again we only change its value to -1, while its position among the keys stays the one of the first time we saw it.

Test set

Case	s	Expected	Got (1)	Got (2)
unique at the start	"carbonara"	0	0	0
unique in the middle	"arrabbiata"	6	6	6
no unique character	"carbonaracarbonara"	-1	-1	-1
unique at the end	"carbonaracarbonara!"	18	18	18
empty string	""	-1	-1	-1
one character	"c"	0	0	0

FINAL QUESTION

Give some potential usage of these LAB 1 developed algorithms in your future semester project.

The professor has not yet explained in detail what the semester project is about and what we will have to do, so we just list the ways in which we could reuse the exercises of this lab:

- $n \bmod 10$ and $n \div 10$ as in exercise 1: $n \bmod 10$ gives the last digit of a number and $n \div 10$ removes it, so they are useful for every operation that needs to work on the digits of a number one by one;
- the stack of exercise 2, to validate input files: for example in a JSON file every { and [must be closed in the right order, exactly like the brackets of the exercise; the stack is also used in the depth-first search on graphs, where it keeps the nodes still to visit, so the last node added is the first one explored;
- merging intervals of exercise 3, for calendars and bookings: every booking is a time interval, so to know when a room is busy we merge the overlapping ones; the same algorithm works with ranges of IP addresses or memory addresses, because a block of addresses is an interval of numbers and overlapping blocks can be merged into one;
- Horner's method of exercise 4, to convert a string of digits into a number: "513" = $(5 \times 10 + 1) \times 10 + 3$, which is Horner with $x = 10$; it is the opposite of exercise 1: there mod and div split a number into its digits, here Horner puts the digits back together;
- the rotation of exercise 5, for round-robin, for example the turns between players in a game or between processes that share the CPU: after the last one it starts again from the first one, so it is like rotating the array;
- the dictionary of exercise 6, to find or remove duplicates: the dictionary counts how many times each element appears, so an element is a duplicate when its count is greater than 1, which is the same check of the exercise but with the opposite condition.